

OOPJ Day 3

Wed, 09 Sept 26

- Three types of programming languages discussed: procedural, object-based, object-oriented
- Procedural (e.g. C): entire business logic concentrated in global functions; functions scattered, no boundary, anyone could call them
- Object-oriented: business logic revolves around objects; everything done via object methods
 - To invoke a method, you call it on an object (e.g. calculator object)
 - Results stored inside the object; accessed via getter methods (e.g. getResult)
- Qualifying a language as object-oriented:
 - Must support all five major features of the OOP paradigm
 - Major features: encapsulation, abstraction, inheritance, polymorphism, modularity
 - Minor features (optional, not compulsory): persistence/serialization, concurrency, RTPI/RTCI/Reflection API
 - Object-based languages do not support all five major features; hence not fully OOP
- Object-based vs. object-oriented distinction:
 - Example: Visual Basic is object-based and does not support inheritance
 - But this does not mean all object-based languages lack inheritance specifically; they may lack any one or more of the five major features
 - An object-based language tomorrow might support inheritance but not polymorphism

Object Definition

- Standard definition (expected in any exam or interview): a real-time entity that has:
 - An identity (can be uniquely identified)
 - A state (set of properties describing the object)
 - Behaviors (set of methods/functions the object can perform)
- Example: a calculator is an object
 - It is a real-time entity
 - It has state (current stored value, memory)
 - It has behaviors (add, subtract, multiply, divide, getResult)

Key OOP Features Explained

- Encapsulation and data hiding:
 - Hiding is not possible without encapsulation; they work in tandem
- Persistence vs. serialization:

- Persistence: stores only the data of the object
- Serialization: stores both the data and the metadata of the object
 - Metadata includes: class name, class structure, data member names, data member types
- Deserialization: reverse process; reads metadata and data from file and reconstructs the object
- Concurrency:
 - An object performing more than one task at any one given instance
- RTPI / RTCI / Reflection:
 - Three related but increasingly advanced concepts
 - Reflection API is Java's highest-level implementation of this concept
 - All three are enhancements of the same underlying idea

Name Mangling and Function Overloading

- Why C does not support function overloading:
 - In C, the compiler renames every function using only the function name (name mangling)
 - Parameters are not considered during name mangling in C
 - So two functions named printf with different parameters would both be mangled to "printf"; no uniqueness possible
- Why C++ and Java support function overloading:
 - In C++, the compiler mangles the name using both the function name and its argument types
 - Example: printf with one int argument → mangled to something like printf_d1; with two arguments → printf_d1d1; etc.
 - Each overloaded version gets a unique mangled name, so the compiler can distinguish them
- Java uses the same name mangling principle as C++ (Java is derived from C++)
- Answer for any interview question on how function overloading works: "The compiler uses name mangling to uniquely identify every function using the function name and its argument types"

Java History and Architecture

- Created by James Gosling (Sun Microsystems)
- Goal: make applications platform-independent
- First application target: set-top boxes (embedded systems)
 - Analysis showed approximately 200 microprocessors would be in an American house
 - Set-top boxes were newly introduced in the US in the 1990s and needed apps for descrambling satellite signals and running games
 - First successful Java application for set-top boxes completed in 1992
- Java 1.0 released in 1994

- Slow adoption despite platform independence
- Two reasons for poor performance:
 - Extra JVM layer inserted between application and OS; communication path: app → JVM → OS → hardware
 - JVM interpreted bytecode line-by-line on every single execution; conversion from bytecode to machine code repeated each time the app ran
- Bytecode explained:
 - Java source code compiled by the Java compiler into bytecode (platform-independent intermediate code)
 - Bytecode stored in .class files
 - Only the JVM can understand and execute bytecode
 - Bytecode is what makes Java platform-independent; it is not machine-specific
- Java 2 released in 1998: major paradigm shift
 - Introduced JIT (Just-In-Time) compilers as part of the JRE
 - On first execution: JIT converts entire bytecode to machine-dependent code and stores it as a temporary file
 - On all subsequent executions: JVM runs the already-converted machine code directly from the temp file
 - Result: conversion overhead occurs only once; performance improved significantly
 - First execution still has a lag; all subsequent executions are fast
- Version naming history:
 - Originally called J2SE (Java 2 Standard Edition), J2EE, J2ME
 - At a Sun ONE conference, someone pointed out the inconsistency of calling version 4.0 or 5.0 still “J2”
 - Sun Microsystems renamed to JSE (Java Standard Edition), JEE, JME
 - Anyone still using “J2SE” today is considered to lack proper knowledge of Java
- Version code names (animal/mammal themed):
 - Java 5.0: Tiger
 - Java 6.0: Dolphin
 - Java 7.0: Mustang
 - Code names dropped from Java 8.0 onwards to reduce confusion among programmers
- Java 8: longest-surviving and most widely used version
- Management history:
 - Up to Java 5.0: managed by Sun Microsystems
 - From Java 6.0 onwards: management transferred to Oracle
 - From approximately 2014: Java officially part of Oracle
 - Oracle now releases a new major version every year (Java 26 current, Java 27 next year, etc.)
 - A major version increment requires significant feature and architecture changes

Java Editions

- Java SE (Standard Edition):
 - Defines the basic language concepts and architecture of a Java program
 - Supports:
 - Console-based applications (CUI: Character User Interface)
 - Graphical-based applications (GUI: Graphical User Interface)
 - Network-enabled applications
 - Data-enabled applications (e.g. via JDBC: Java Database Connectivity)
 - Categorized into two components: JDK and JRE
 - JDK (Java Development Kit): set of tools, libraries, and debugger required to develop, execute, and debug a Java program on the underlying platform
 - JRE (Java Runtime Environment): set of tools and libraries required only to execute a Java program
 - To develop: need JDK; to only run: need JRE
 - JDK downloaded from oracle.com
- Java EE (Enterprise Edition):
 - A set of specifications (not full implementations) defining how distributed applications in Java should be created
 - Specifications explained: define the skeleton/rules (class names, method names, expected behavior) but not the business logic implementation
 - Example analogy: told to create a Calculator class with add, subtract, multiply, divide methods and what they should do, but not how to implement the logic
 - Distributed applications also called n-tier based applications
 - Key technologies within Java EE:
 - Servlets
 - JSP (Java Server Pages)
 - JSF (Java Server Faces)
 - JNDI (Java Naming and Directory Interface)
 - JMS (Java Messaging Service)
 - EJB (Enterprise Java Beans)
 - JPA (Java Persistence API)
 - Java Mail API
 - JTA (Java Transaction API)
- Java ME (Micro Edition):
 - Used for developing applications for portable or handheld devices
 - Target devices: mobile phones, PDAs (Personal Digital Assistants), Blu-ray players, set-top boxes
 - Development now very limited; not a major focus area
 - Reason: Android and iOS dominate the mobile space
 - Historical note: Android was originally built on Java ME libraries; Google later built Android on Linux and it became a massive success
 - Sun Microsystems tried to launch Java ME broadly but it did not succeed before Google's Android took over

Java Data Types and Memory Model

- Java picked up basic principles from C and C++
- Two categories of data types in Java:
 - Primitive data types (language inbuilt)
 - Object types (instances of user-defined or library classes)

Primitive Data Types

- Sizes are fixed across all platforms (unlike C/C++ where sizes varied by architecture):
 - short: 2 bytes
 - int: 4 bytes
 - long: 8 bytes
 - float: 4 bytes
 - double: 8 bytes
 - boolean: 1 bit
 - byte: 1 byte
 - char: 2 bytes (explained below)
- Why fixed sizes matter: Java is platform-independent; if sizes changed per platform, the same program would behave differently on different machines
- char is 2 bytes in Java (unlike 1 byte in C/C++):
 - C/C++ char supports 0–255 characters (ASCII), sufficient for English only
 - Java uses the Unicode character set (Universal Coding Standard): 2 bytes = $2^{16} = 0-65,535$ characters
 - Supports multilingual characters: English, Hindi, Marathi, Gujarati, Punjabi, Malayalam, Japanese, etc.
 - This concept is called the Unicode character set
- var keyword: size determined at runtime based on what data type it refers to; cannot state a fixed size for var

Why Java Is Not 100% Object-Oriented

- Rules for a 100% OOP language:
 - Everything must be treated as an object
 - All major OOP features must be supported
 - Nothing should be global (not even functions)
- Java fails rule 1: primitive types (int, float, char, etc.) are not objects; they directly hold values and cannot have methods called on them
- Java is close to 100% OOP but not fully there because of primitive type support
- Wrapper classes bridge this gap:
 - For every primitive type, Java provides a corresponding class: Integer, Float, Double, Character, Boolean, Short, Long, Byte
 - These classes wrap the primitive type as a data member and add utility methods

- Using wrapper classes instead of primitives makes a program 100% object-oriented
- Key utility method: parse methods (parseInt, parseFloat, parseDouble, parseLong, etc.) convert String to the corresponding primitive type

References and Memory

- In Java, a variable of any class type is called a reference (not an object)
- References in Java are equivalent to pointers in C:
 - They hold the memory address of an object
 - They cannot be arithmetically manipulated (e.g. ref++ is not allowed)
 - This restriction exists for security reasons
- References in Java are different from reference variables in C++:
 - C++ reference variables are aliases (another name for an existing variable)
 - Java references are memory addresses, like C pointers
- Arrays in Java are objects:
 - Must be allocated on the heap using the new operator
 - Syntax: `byte[] arrInput = new byte[100];`
 - This creates a reference (arrInput, 4 bytes) pointing to an array object on the heap with 100 byte-sized elements
 - Contrast with C/C++: `int arr[100]` allocates everything on the stack in one step; Java always separates reference from object
 - args (the array parameter in main) is a reference to the array object; args[0] is the first element of that array (a String object)

Eclipse IDE Setup

- Eclipse is open-source, originally started by IBM, now managed by the Eclipse Foundation
- Open source meaning: anyone can suggest contributions; the language/tool developer decides whether to accept them; source code cannot be directly modified by users
- Eclipse is itself written in Java, making it technically platform-independent
- The eclipse.exe launcher is a small C++ executable (~546 KB) that starts the JVM and loads the main Java application
- Memory footprint at runtime: approximately 450–500 MB RAM

Downloading Eclipse

- Go to eclipse.org → Download Packages
- Choose “Eclipse IDE for Enterprise Java and Web Developers” (not the basic Java Developers edition)
 - Reason: the course covers both Java SE now and Java EE later; this edition supports both
 - Also supports JavaScript, web scripting, and other languages

- Select Windows x86-64 download; file is a zip of approximately 550 MB
- Extract the zip; place the eclipse folder in C:\eclipse
- No installer needed; run eclipse.exe directly from the extracted folder

First Launch and Workspace Setup

- On first launch, Eclipse asks for a workspace location
 - Workspace: the environment where all projects are stored
 - Create one workspace; all projects for the course go inside it
 - Can check “use as default” to skip this prompt on future launches
- First load takes significant time; Eclipse loads a large amount into memory
- Eclipse plugin marketplace: eclipse.org/marketplace; over 1,200 plugins available under the tools section alone
- Flavors of Eclipse available for different languages: Java, Java EE, C/C++, PHP, Scala, embedded C/C++, DSL, RCP, etc.
 - Advantage: same IDE look and feel across languages; no need to learn a new IDE when switching languages

Creating a Java Project

1. Ctrl+N or click New → select Java Project → click Next
2. Enter project name (e.g. “FirstProgram”)
3. Select execution environment JRE: choose Java 17
 - Features used in this course go up to Java 17; no need for 21, 22, or 26
4. Uncheck “Create module-info.java” (modules introduced in Java 10+; not covered yet)
5. Click Finish → when prompted, select “Open Java Perspective”
6. In Package Explorer (left panel): expand project → src folder → right-click src → New Class
7. Name the class (e.g. “Program”); check the option to include a public static void main stub
8. Eclipse auto-generates the class with the main method signature

Compilation, Running, and Debugging

- Ctrl+S saves and auto-compiles; compilation errors appear immediately in the Problems panel
- Double-click any error in the Problems panel to jump to the exact line in the editor
- Run: Ctrl+F11; output appears in the Console panel at the bottom
- Debug: F11; set breakpoints by double-clicking the line number margin (a small circle appears)
 - On first debug run, Eclipse asks to switch to Debug Perspective; say Yes
 - Debug Perspective shows Variables panel on the right: inspect array contents, sizes, values at runtime

- F5: Step Into (enters a function call line by line); F6: Step Over (executes the function and moves to next line)
 - Stop button returns to Java Perspective
- To pass command-line arguments in Eclipse: click the arrow next to the play button → Run Configurations → Arguments tab → enter values separated by spaces
- Auto-build can be disabled: Project menu → uncheck “Build Automatically”

Project Management Best Practice

- Close projects when finished: right-click project → Close Project
- Reason: all open projects load into memory on Eclipse startup and recompile on every save
- With 40–50 projects accumulated over the course, leaving them all open will drastically slow Eclipse

Environment Variable Setup (Command-Line Use)

- The command prompt searches for executables in: current directory first, then PATH environment variable entries
- If javac is not found, it means the JDK bin folder is not in PATH
- Temporary fix (session only): set PATH=C:\Java\jdk\bin;%PATH%
 1. %PATH% appends the existing PATH so other tools are not lost
 2. Disappears when the command prompt is closed
- Permanent fix:
 1. Right-click This PC → Properties → Advanced System Settings → Environment Variables
 2. Under System Variables (not User Variables; system applies to all users, user applies only to current login): find Path → Edit → New → add C:\Java\jdk\bin → move it to the top of the list
 3. Priority matters: other software (e.g. Oracle Database) may bundle an older Java version; putting JDK bin at the top ensures the correct version is found first
 4. Also create a new System Variable: JAVA_HOME = root JDK folder (parent of the bin folder)
 5. Click OK to save; changes are permanent

Writing and Running Java Programs

Hello World (Notepad / Command Line)

- Create a file entry.java in a folder (e.g. E:\cargo)
- Define a class named entry (class name and file name do not have to match at this stage)
- Inside the class, define: public static void main(String[] args)

- public: access specifier
- static: allows JVM to call main directly on the class name without creating an object
- void: no return type
- main: required method name; JVM looks specifically for this name
- String[] args: array of the predefined String class; receives command-line arguments
- Why static: JVM needs to call main without knowing whether the programmer wants an object created; static allows direct invocation as `ClassName.main(...)`
- Print output: `System.out.println("Hello World")`
 - System: predefined class representing the underlying system (the JVM in this context)
 - out: object of System for output
 - println: method that prints and moves to the next line
- Compile: `javac entry.java` → generates `entry.class` (named after the class, not the file)
 - One .class file generated per class defined in the source file
- Run: `java entry` (class name only, no extension)
 - Providing `entry.class` as the argument causes an error: JVM interprets “entry” as a namespace and “class” as the class name

Command-Line Arguments Program

- Command-line arguments received as String array args
- `args[0]` = first argument, `args[1]` = second argument, etc.
- JVM ignores the “java ClassName” part and passes only the remaining tokens as args
- Problem: `args[0] + args[1]` with two numeric strings (e.g. “10” and “20”) produces “1020” not 30
 - Plus operator on strings performs concatenation, not addition
- Solution: parse strings to integers using `Integer.parseInt(args[0])`
- Demonstrated via debugger: Variables panel shows args array with size 2, elements “10” and “20” as strings

Console Input Program

- Java accepts console input as bytes via `System.in.read(byte[])`
- Steps to accept and process console input:
 1. Declare a byte array: `byte[] arrInput = new byte[100];`
 2. Print prompt: `System.out.println("Enter the value:");`
 3. Read input: `int length = System.in.read(arrInput);`
 - Returns the number of bytes read, including the two trailing characters
 4. Remove trailing `\n` (newline) and `\r` (carriage return) appended automatically on Enter:
 - Create a new array: `byte[] arrFinal = new byte[length - 2];`
 - Copy contents: `System.arraycopy(arrInput, 0, arrFinal, 0, length - 2);`

5. Convert byte array to String: `String objString = new String(arrFinal);`
 6. Parse to required type: `int num = Integer.parseInt(objString);`
- This same code works for any data type input; only the parse method changes
 - `System.in.read` throws `IOException`; must be wrapped in try-catch
 - `NumberFormatException` thrown if input cannot be parsed (e.g. "55a" passed to `parseInt`)

Calculator Program and OOP Design Progression

Stage 1: Inline Code (Bad Practice)

- Declared `int num1, num2, result`; assigned values directly; printed result
- Demonstrated type safety: Java will not compile if a local variable is used before being initialized (unlike C++ which shows garbage values)
- Distinction between initialization (value set at point of declaration) and assignment (value set after memory already allocated)

Stage 2: Copy-Paste for Two Inputs (Disaster)

- Repeated the entire console input block twice for `num1` and `num2`
- Problems with copy-paste:
 - Code size increases drastically; maintenance becomes complex
 - A bug in the repeated block must be fixed in every copy (e.g. if `length - 2` was wrongly written as `length - 1`, it must be corrected in 20 places if copied 20 times)
 - Rule: whenever you use `Ctrl+C` and `Ctrl+V` in code, remember one word: disaster

Stage 3: Extracted Methods (Better)

- Moved console input logic into a separate static method: `getInt()`
 - Returns `int`; handles all byte array creation, reading, trimming, parsing
 - Returns `-1` as default if an exception occurs
- Added `getFloat()` similarly: same logic, uses `Float.parseFloat` instead
- Problem: `getInt` and `getFloat` still duplicated the byte array and reading logic

Stage 4: Abstracted `getString()` (Best)

- Observation: regardless of desired data type, input always arrives as a `String`
- Created `getString()` method: handles all byte array creation, reading, trimming, and `String` conversion; returns a `String`
- `getInt()` now simply calls `getString()` and calls `Integer.parseInt` on the result; one-liner
- `getFloat()` now simply calls `getString()` and calls `Float.parseFloat` on the result; one-liner
- Demonstrates: modularity, reusability, abstraction (generalization of the common input logic)

Stage 5: Separated into Proper Classes (OOP)

- Program class should only define the entry point and control flow; not responsible for input or computation
- ConsoleInput class: sole responsibility is accepting input from the console
 - Contains getString(), getInt(), getFloat() as static methods
- Calculator class: sole responsibility is performing arithmetic
 - Contains add(int, int), add(float, float), subtract, multiply, divide methods (non-static, called on an object)
 - Methods overloaded for int and float versions
- Program class: creates a Calculator object; calls ConsoleInput.getFloat() for inputs; calls calculatorObj.add(num1, num2) for result
- Classes in the same project can be used directly without any import-like mechanism (dynamic linking in Java)

Assignments for Day 3

- **Install Eclipse IDE for Enterprise Java and Web Developers** from eclipse.org
 - Select JRE 17, uncheck module-info.java on project creation
 - **Write and run Hello World in Eclipse**
 - Get comfortable with the IDE, auto-compilation, Problems panel, and Console output
 - **Implement the command-line argument program**
 - Pass two numbers, observe string concatenation vs. integer addition behavior
 - **Implement the full Calculator program in Eclipse**
 - Build incrementally: start with inline code, then extract methods, then separate into ConsoleInput and Calculator classes
 - Do not jump directly to the final version; type each stage to understand the progression
 - **Overload add, subtract, multiply, and divide in the Calculator class**
 - Implement both int and float versions of each method
 - **Submit team name and lead name to Vipul by Friday 11th September**
 - Required to receive alumni mentor contact details; team size is 10 members
 - **Update Zoom display name** to format: last three digits of PRN number, hyphen, name, hyphen, easy charge identifier
-